Paper properties against development theorems

Source: **C. A. Gunter and D. S. Scott, "Semantic Domains,"** *Handbook of Theoretical Computer Science* Vol. B, 1990, pp. 633–674.

The development holds **1308 theorem-ish declarations** for a paper with **30 numbered results**. That ratio is the question this file exists to make answerable. It is not answerable from the ratio alone, because the paper's 30 numbered results are not 30 assertions — several are conjunctions over an operator list — and because a formalization must state things the paper assumes.

All figures below are measured by `scripts/counts.sh`, `scripts/module-counts.sh` and `scripts/unused-theorems.sh` at commit `628181e` (r0037 merged), not estimated.

1. What the paper actually asserts

A **property** here is one atomic assertion: a conjunct of a numbered result, not the numbered result itself. Lemma 17 is one numbered result and ten properties.

#	§	Result	Properties	Note
1	2	Thm 1, Thm 2, Thm 3	3	one each
2	3	Lem 4	4	the paper's own four parts
3	3	Lem 5	2	compacts of `im(p)`; `im(p) ∩ K(D) ◁ K(D)`
4	3	Thm 6	1	the correspondence
5	3	Thm 7	4	cpo, bounded complete, algebraic, countably based
6	4	Lem 8	4	four isomorphism laws
7	4	Lem 9	6	six isomorphism laws
8	4	Lem 10	7	seven operators
9	4	Thm 11	2	the theorem and its converse
10	4	Thm 12	6	three theories `T♮`/`T♯`/`T♭`, each existence **and** uniqueness
11	4	Lem 13	2	`D♭`, `D♯`; there is no convex conjunct
12	4	Thm 14	2	two directions of a characterization
13	6	Prop 15, Thm 18, Lem 19, Lem 20	4	one each
14	6	Thm 16	2	algebraic lattice; the embedding conjunct
15	6	Lem 17	10	ten operators
16	7	Thm 21, Thm 22, Lem 23, Lem 24, Thm 25, Thm 26, Thm 27	7	one each
17	7	Lem 28	9	nine operators
18	7	Thm 29	2	two sentences, different evidential status
19	7	Lem 30	10	ten operators
—	—	**numbered total**	**87**	across 30 numbered results
20	—	unnumbered prose claims	12	`PaperInventory.md` row 3
—	—	**paper properties, total**	**99**	

Definitions are excluded — there are about 13, and they are objects rather than assertions.

Caveat on this tally. The conjunct counts are read off `PaperInventory.md`, not re-derived from the PDF. Three of them have already moved once: Lemma 28 went 7 → 9, Lemma 30 went 9 → 10, and Lemma 17 went 5 → 10 when the dropped glyphs were recovered. Re-deriving the whole column from the rendered pages is the first task of the audit plan.

2. What the development holds

#	Kind	Count
1	`theorem` / `lemma`	**1308**
2	of which `@[simp]`-tagged	139
3	`def` / `abbrev`	398
4	`instance`	94

#	Kind	Count
5	modules	72
6	lines	27892

Ratios. 1308 / 99 ≈ **13.2 theorems per paper property**; 1308 / 30 ≈ 43.6 per numbered result.

3. Where the mass is

The twenty largest modules hold 806 of the 1308, 62%.

#	Module	Thms	Serves
1	`Colimit.lean`	73	`V` as the ω-colimit — Thm 29
2	`ContinuousAlgebra.lean`	62	Thm 12, all three theories
3	`PRepSum.lean`	62	Lem 28's sums, `Lemma28AtU`
4	`Dyadic.lean`	59	`U` and Thm 27
5	`Atomless.lean`	56	Thm 27's Boolean-algebra step
6	`PRepFun.lean`	52	Lem 28's function spaces
7	`IdealCompletion.lean`	44	Thm 11
8	`Skeleton/Sum.lean`	43	Lem 10's and Lem 17's `⊕`, `⊗`
9	`Combinator.lean`	42	Thm 26
10	`FinitaryProjectionPoset.lean`	42	Thm 16, Lem 20
11	`PRep.lean`	39	Lem 28's statement and scheme
12	`LemThirty.lean`	37	Lem 30, Thm 29's second sentence
13	`ContinuousConstruction.lean`	35	Thm 18 (r0031 route, now superseded)
14	`MinimalUpperBounds.lean`	30	Thm 18's `U^∞` machinery
15	`CombinatorRep.lean`	29	Lem 28 at the closure reading — **superseded**
16	`FinitaryProjectionEmbedding.lean`	28	Thm 16's refutation
17	`BifiniteUniversal.lean`	26	Thm 29's first sentence
18	`Powerdomain/Plotkin.lean`	24	the convex powerdomain
19	`JungSFP.lean`	23	Thm 18 steps 2–3
20	`JungFinite.lean`	22	Thm 18 step 4 and assembly

Rows 13 and 15 are the visible candidates for retirement: `ContinuousConstruction` implements r0031's route to Theorem 18, which r0036 measured as *equivalent* to Theorem 18 rather than below it and which Jung's proof never passes through; and `CombinatorRep` proves Lemma 28's conjuncts at the closure reading, which r0037 kernel-checked as not transferring to the projection notion the paper means. Together they are 64 theorems. **Neither is proposed for deletion here —** the `⊗`/`⊕` counterexample lives in `CombinatorRep` and is load-bearing evidence — but both are exactly what the audit must classify.

4. Theorems nothing cites

`scripts/unused-theorems.sh`: **130 of 1250 distinct names, about 10%**, occur only at their own declaration.

That is a candidate list, not a defect count. Three populations are mixed in it, and r0020's hand audit over the then-37 modules found all three:

#	Population	Example from the current list	Correct action
1	**Terminal by design —** the paper's own claims, which nothing *should* cite	`theorem6`, `injective_embedding`, `surjective_projection`	keep; they are the deliverable
2	**Projection/`simp` API —** `_apply`, `_coe`, `_bot` equations that `simp` may fire without naming	`apHom_apply`, `liftMap_coe`, `smashEmbed_bot`	keep if `simp`-tagged and firing; otherwise review
3	**Speculative API —** written for a caller that never appeared	r0020 found 6 and commented them out in place	comment out with a note, as r0020 did

The script under-reports rather than over-reports: it matches on the final name component, so two `map_bots` in different namespaces mask each other.

5. The honest reading

13.2 theorems per paper property is **not by itself evidence of bloat**. A formalization must supply what the paper assumes, and this paper assumes a great deal — `PaperInventory.md` records that the whole $\ll$ calculus, the `compactsBelow` machinery, the pointwise order on `D → E` and the step-function adjunction are elided in the text and had to be built. Theorem 7 alone is six of the twelve prose claims.

What *would* be evidence of bloat, and what the audit must find, is any of:

1. a theorem in population 3 above — API with no caller and no paper claim;
2. two theorems with the same statement under different names (r0028 shipped one such pair, invisible to `lake build` until an axiom audit imported both);
3. a module whose result has been superseded, kept whole rather than reduced to the part that is still evidence — rows 13 and 15 are the candidates;
4. support proved at a strength nothing consumes.

The plan for that audit is `plans/r0038-plan-from-orchestrator-to-orchestrator-theorem-audit.md`.